SolveIt

The Complete Solution Guide

rustSolveIt, on pure-Rust SUNDIALS 7.8.0

17 August 2026

Written for a reader who has never used this program, has never run a physics simulator, and does not write Rust. Every term is defined the first time it is used. Every number printed below was produced by the program on the machine that wrote this file; nothing is quoted from memory.

Contents

1 What this is, in one page

SolveIt simulates classical mechanics and one-, two- and three-dimensional quantum mechanics. You describe a situation — some bodies, some fields, maybe a rigid box to put them in — and it tells you what happens next, to as many digits as the arithmetic allows.

You talk to it by typing short commands, one per line:

```
In[1]:= new sphere { mass = 2, radius = 0.5, position = [0, 10, 0],
                     velocity = [1, 0, 0] }
Out[1]= obj0
In[2]:= set system.gravity = [0, -9.81, 0]
In[3]:= step 1
Out[3]= t = 1 (advanced by 1, 12 solver steps)
In[4]:= get obj0.position
Out[4]= [1, 5.095000000000006, 0]
```

That is the whole interface. No configuration files, no build system to learn, no graphics API.

The one thing that makes this different from a game engine. Game physics is allowed to be approximately right — it only has to look plausible for the next sixteen milliseconds. This program's job is to be *measurably* right. Every trajectory it prints comes out of a production-grade differential-equation solver with genuine error control, and every claim in the documentation is a number you can reproduce by running the command underneath it.

Concretely, and these are the actual measured values:

claim	**measured**
the outer solar system, integrated for 1,369 years	Pluto's position matches the reference to 8 decimal places; total energy drifts by 7.8 parts in 10^7
a torque-free tumbling body	angular momentum changes by **exactly zero**
a ball dropped on a plate	the moment of impact is right to 1 part in 10^{16}
three shapes rattling in a rigid box, 36 collisions	energy conserved to 3 parts in 10^{16}
Kepler's third law across four orbits	T^2/a^3 identical to the last bit for all four

1.1 The rules the code lives under

1. **Everything is integrated by SUNDIALS.** There is no hand-written stepper anywhere in the program — not in the examples, not in the graphics playback, not as a "fast path".
2. **No `unsafe` code.** Rust's memory-safety checks are on everywhere, with no escape hatches.
3. **No dependencies at all.** Not one package from the internet. The solver library, the web server behind the graphics window, the WebSocket protocol, the SHA-1 implementation, the special-function library — all of it is in this repository.
4. **No compiler warnings.** The build is clean, and it is set up to *fail* if it stops being clean.

Rule 3 has a consequence worth spelling out: `git clone`, `cargo build`, done. No network access during the build, nothing to go stale, nothing that can be pulled out from under you.

2 Installing and running it

You need Rust. Then:

```
git clone https://github.com/once-ere/rustSolveIt_Using_SUNDIALS_7_8_0.git
cd rustSolveIt_Using_SUNDIALS_7_8_0/version-7.8.0
cargo run
```

The last command builds everything and drops you at the `In[1]:=` prompt. Type `HELP` for the reference card, `QUIT` to leave.

how	**command**	**when**
interactive notebook	`cargo run`	you are exploring
batch script	`cargo run -p posim -- --script my_run.posim`	you want it reproducible
dynamic notebook	`cargo run -p posim --release -- --notebook dynamic_notebooks/kepler_orbit.posim`	run a saved session, open its graphics window, keep typing
machine mode	`cargo run -p posim -- --machine`	another program is driving

A "script" is just a text file of the same commands you would type. Comments start with `#`. Every example in §7 is a script in `scripts/solveit/`.

Checking your build: `cargo test --workspace` — 603 tests should pass.

3 The five things you need to know

3.1 `RUN` takes a duration, not a destination

```
In[1]:= run 1.7 steps 170        # advances BY 1.7, so t is now 1.7
In[2]:= run 1.19 steps 120       # advances BY 1.19, so t is now 2.89
```

If you want to stop at an absolute time, subtract. This trips up everyone once. `STEP` *dt* is the same thing with one output point.

3.2 Gravity is on by default, and it is between *bodies*

`system.g_constant` defaults to **1**, which means every pair of bodies attracts every other pair. That is what you want for a solar system and emphatically not what you want for three blocks in a box: two touching surfaces are at nearly zero separation, and the attraction between them is nearly singular.

If you are studying collisions, turn it off: `set system.g_constant = 0`.

The difference is not subtle. The same three-shapes-in-a-box scenario conserves energy to **3 parts in** 10^{16} with $G = 0$, and drifts by **3.2 %** with $G = 1$. Neither number is a bug — they are different physical systems. `system.uniform_gravity` is the separate thing you usually mean by "gravity": a constant downward field, like a laboratory.

3.3 Objects are numbered, and deleting renumbers them

`obj0`, `obj1`, ... in creation order. `DEL` 1 removes `obj1` **and renumbers everything after it**. Give things names instead: `new sphere as earth { ... }`, then `get earth.position`. Names survive deletions of other objects.

3.4 Collisions must be switched on

`COLLIDE ON`. Without it, bodies pass through each other. This is a deliberate default: collision detection changes how the solver picks its steps, and a great many problems (orbits, fields, quantum) never need it.

3.5 $|dE/E|$ is printed for free, and you should read it

Every `RUN` reports the relative change in total energy. For a closed system this should be tiny. If it is not, either the physics genuinely does not conserve energy (a magnetic torque, an inelastic bounce, a driven potential — see examples 4 and 8) or something is wrong with your setup. It is the cheapest sanity check you will ever get.

4 What the program is made of

piece	what it does
`posim`	the command language: reads your line, checks it, compiles it to a tiny stack program, runs it. Also the graphics window and the machine-mode protocol.
`physical_object`	the physics: one `physical_object` type that is simultaneously a point mass, a rigid body and a shaped, collidable solid. Plus all the time integration.
`quantum`	1-D, 2-D and 3-D quantum mechanics: bound states, wavepacket propagation, tunnelling, absorbing boundaries.
`special_functions`	Bessel, Legendre, Hankel, Airy, gamma, Wigner symbols, quadrature, eigenproblems.
`sundials_rs`	the differential-equation solvers (§5).
`jupyter`	a JupyterLab kernel.

4.1 The one type that does everything

The original 2002 SolveIt had three separate types: a point particle, a rigid body, and a 3-D rigid body. Each had its own integrator, its own bugs, and its own idea of what "position" meant. This version has **one** type, `physical_object`, which is the union of all three. A point mass is just a `physical_object` whose shape is `POINT` and whose inertia tensor is zero.

That sounds like a detail. It is why a point mass, a spinning cuboid and a dumbbell can all be in the same collision and the same conservation sum, without a single special case.

4.2 The state vector

Every object carries **13 numbers**:

[position (3) | momentum (3) | orientation quaternion (4) | angular momentum (3)]

Momentum rather than velocity, and angular *momentum* rather than angular velocity, both on purpose: those are the quantities that are conserved, and the ones that behave correctly when you change a mass mid-run. Quaternions are written **w first**: $[w, x, y, z]$.

5 The engine: SUNDIALS 7.8.0 in pure Rust

5.1 What SUNDIALS is

SUNDIALS — SUite of Nonlinear and DIfferential/ALgebraic equation Solvers — is Lawrence Livermore National Laboratory's library of differential-equation solvers, in production use in physics and engineering codes since the 1990s. It is written in C.

`sundials_rs/` in this repository is a **line-for-line translation of SUNDIALS 7.8.0 into pure Rust**. Not a wrapper, not bindings: there is no C compiler involved, no shared library to link, no FFI boundary. It carries the same control flow, the same constants, the same tolerances, and — a subtler point that matters a great deal — the same *order of arithmetic operations*, because floating-point addition is not associative and changing the order changes the answer in the last digits.

5.2 Which solver runs when

you type	what runs
`METHOD ADAMS` (default)	**CVODE Adams–Moulton**: variable order, variable step size, Newton iteration, dense finite-difference Jacobian. The general-purpose choice.
`METHOD BDF`	**CVODE BDF**: backward differentiation formulas, for *stiff* problems — ones with two very different timescales, like a fast magnetic gyration inside a slow drift.
`METHOD SPRK` *table* [*dt*]	**ARKODE SPRKStep**: a *symplectic* partitioned Runge–Kutta method at a fixed step. Only valid for separable systems, and worth it when you have one.

"Adaptive" means the solver chooses its own internal step size to keep the estimated local error under the tolerance you asked for. When it reports `1289 solver steps` for a run you asked to sample at 400 points, those 1289 are its own steps; the 400 are just where it stopped to tell you the answer. **The output cadence does not affect the physics.** That property is tested, not assumed.

5.3 What the 7.8.0 upgrade brought

This repository is `once-ere/rustSolveIt` with its 7.7.0 engine replaced by the 7.8.0 one. Two things came with it.

Four more solver families.

crate	solves	used here?
`cvode_rs`	ordinary differential equations (Adams / BDF)	yes
`arkode_rs`	Runge–Kutta: explicit, implicit, IMEX, multirate, symplectic	yes
`ida_rs`	differential-algebraic systems $F(t, y, \dot{y}) = 0$	yes — `CONSTRAIN` + `METHOD IDA`
`kinsol_rs`	nonlinear algebraic systems, Anderson acceleration	yes — `EQUILIBRIUM`
`cvodes_rs`	CVODE plus forward and adjoint sensitivity analysis	yes — `SENSITIVITY`
`idas_rs`	IDA plus sensitivity analysis	yes — `SENSITIVITY` while constra

All six are compared byte-for-byte against the output of the original C programs.

5.4 The three questions that are not "what happens next"

Four joints. `CONSTRAIN` *a b* is a rod (1 row, 5 freedoms left); `BALL` *a b* shares a point (3 rows); `UNIVERSAL` *a b u w* shares a point and keeps two shafts square, a Cardan joint (4 rows); `HINGE` *a b axis* shares a point *and* an axis, leaving exactly one freedom (5 rows) — a door, a knee, a pendulum. The last three grip orientation, which is why `METHOD IDA` carries the full 13-numbers-per-object state.

A rigid rod, held exactly — `CONSTRAIN` *a b* then `METHOD IDA`. A rod is a geometric fact, not a very stiff spring: the bob is *exactly* L from the pivot, always. Saying so turns the equations of motion from an ODE into a differential-algebraic equation, which is what IDA solves. A body with `inverse_mass = 0` becomes an **anchor** — it never moves and absorbs the reaction — so anchor + bob + rod is a pendulum. `CONSTRAINTS` reports how far the rod has strayed; over a 20-second swing it stays at roundoff.

Where it comes to rest — `EQUILIBRIUM`. Not an integration: KINSOL solves for a configuration where every free body has zero net force, every anchor is where it started and every rod is the right length, then puts the bodies there and stops them. It says nothing about *stability* — a pencil on its point is an equilibrium too.

How much the answer depends on an input — `SENSITIVITY 3 "gravity.y"`. Running twice with slightly different inputs and subtracting throws away most of your digits. This integrates the derivative alongside the state instead. CVODES for an ordinary system; IDAS when a `CONSTRAIN` is active.

A more faithful API. The 7.8.0 translation models C's opaque pointers directly rather than smoothing them into idiomatic Rust. Vector data is reached through a borrow guard, constructors return `Option` where C returns a possibly-null pointer, and destructors blank the handle the way `free(p); p = NULL;` does. None of that is visible from the command language; the full call-by-call table is in `PORT_7.8.0_PROVENANCE.md`.

And the physics did not move. All six self-checking physics examples, all twelve collision scripts and all 59 dynamic notebooks produce output that is byte-for-byte identical under 7.7.0 and 7.8.0.

6 How collisions actually work

6.1 The problem with checking every frame

The obvious way to detect a collision is to advance time a little, then ask "are these two objects overlapping?" If yes, back up and resolve it.

This fails in a specific, embarrassing way. Consider a bullet at 100 m/s and a plate 5 mm thick. In one hundredth of a second the bullet moves a full metre — two hundred times the plate's thickness. Check before the step: not overlapping. Check after: not overlapping. The bullet is now on the other side, and the program never noticed. This is called **tunnelling**.

6.2 What this program does instead

It hands the solver a **continuous function** — the signed gap between two surfaces, positive when apart, negative when overlapping — and asks it to find the moment that function crosses zero. This is **rootfinding**, a first-class feature of CVODE, not something bolted on top.

Three things follow. **The impact time is exact**, not quantised to a frame: the solver interpolates its own solution back to the crossing. **The solver caps its own step size** while rootfinding is armed, computed from the smallest feature among the paired bodies divided by the fastest surface approach speed — and it accounts for *acceleration*, so a body released from rest above a thin plate still gets a finite cap. **Systems with nothing to collide are unaffected**: with no collidable pairs the code path is bit-identical to running with collisions off, and that is a tested property.

6.3 What happens at the moment of contact

The solver stops at the crossing and hands control back. The program then reads the state at exactly that instant; computes the contact point and normal from the two shapes' actual geometry (a *support function*, not a bounding sphere — see example 13); applies an impulse along the normal, sized by the two bodies' masses, inertias and coefficients of restitution; and re-initialises the solver at the new state. The impulse is applied at *one shared point* for both bodies, which is what makes angular momentum come out right for off-centre hits.

6.4 The Zeno problem

A bouncing ball with restitution below 1 makes infinitely many bounces in finite time. The program counts events per *burst* — a run of events with no real flight between them. Past a threshold it forces the remaining contacts plastic; past twice the threshold it projects out any overlap and disarms rootfinding for the rest of the output interval. The counting is per-burst rather than per-output-interval, because per-interval counting made the physics depend on how often you asked for output.

7 Nineteen worked examples

Every transcript below is genuine program output. The scripts are in `scripts/solveit/`:

```
cargo run -p posim --release -- --script scripts/solveit/01_elastic_head_on.posim
```

7.1 A head-on elastic collision between unequal masses

Two spheres, masses 3 and 1, approach head-on at $+2$ and -4. Elementary mechanics gives $v_1' = ((m_1 - m_2)u_1 + 2m_2u_2)/(m_1 + m_2)$ and the mirror image for v_2'. The answer is exact, and both energy and momentum must survive the impulse untouched.

```
In[5]:= energy
Out[5]= 14
In[6]:= momentum
Out[6]= [2, 0, 0]
In[7]:= run 2 steps 200
Out[7]= t = 2 (53 solver steps, 200 snapshots, |dE/E| = 0.000e0,
        1 collision(s))
In[8]:= get heavy.velocity
Out[8]= [-1, 0, 0]
In[9]:= get light.velocity
Out[9]= [5, 0, 0]
In[14]:= ((m1 - m2) * u1 + 2 * m2 * u2) / (m1 + m2)
Out[14]= -1
In[15]:= ((m2 - m1) * u2 + 2 * m1 * u1) / (m1 + m2)
Out[15]= 5
In[16]:= energy
Out[16]= 14
In[17]:= momentum
Out[17]= [2, 0, 0]
```

-1 and 5 exactly, not approximately. Energy 14 before and 14 after; momentum $[2, 0, 0]$ before and after. And $|dE/E| = 0$ — the run lost nothing at all. Note that `LET` makes a session variable and a bare expression line is evaluated and printed, so the closed form is checked *inside the language*.

7.2 Kepler's third law, four orbits at once

For a circular orbit of radius a about mass M, the period is $T = 2\pi a^{3/2}/\sqrt{GM}$. Kepler's third law says T^2/a^3 is the same number for every orbit around the same central mass. Each planet is run for its own period and must come home.

```
Out[5]=  [1.0000000044544244, 0, -0.00000014431704032681625]
Out[11]= [2.000000000309673,  0,  0.00000002604593958739631 6]
Out[17]= [3.0000000004230944, 0,  0.000000040718812885712 22]
Out[23]= [4.000000000077065,  0,  0.000000072487458217774 47]

In[24]:= 0.19869176531592203 * 0.19869176531592203 / 1
Out[24]= 0.039478417604357434
In[25]:= 0.5619851784832581 * 0.5619851784832581 / (2 * 2 * 2)
Out[25]= 0.039478417604357434
In[26]:= 1.0324326977181857 * 1.0324326977181857 / (3 * 3 * 3)
Out[26]= 0.039478417604357434
In[27]:= 1.5895341225273762 * 1.5895341225273762 / (4 * 4 * 4)
Out[27]= 0.039478417604357434
```

All four are the same 17 digits. Not "close" — identical. That is $4\pi^2/1000$.

7.3 Bound, parabolic and hyperbolic, decided by a sign

The shape of an orbit is decided by the specific orbital energy $\varepsilon = v^2/2 - GM/r$: negative is an ellipse, exactly zero a parabola, positive a hyperbola. The escape speed at r is $\sqrt{2GM/r}$; at $r = 4$ with $GM = 1000$ that is $\sqrt{500}$.

```
In[7]:= 20 * 20 / 2 - gm / 4
Out[7]= -50
In[8]:= 22.360679774997898 * 22.360679774997898 / 2 - gm / 4
Out[8]= 0.00000000000002842170943040401
In[9]:= 24 * 24 / 2 - gm / 4
Out[9]= 38
```

-50, then 2.8×10^{-14} (the closest a 64-bit float gets to zero after squaring an irrational number), then $+38$. Three conics from one formula.

7.4 The restitution ladder, and honest energy loss

A ball with restitution e reaches $e^{2n}h$ after n bounces. The centre starts at 5.5 with radius 0.5, so it falls 5.0; $g = 10$, $e = 0.7$. `RUN` **takes a duration**, so the three runs are 1.7, 1.19, 0.833.

```
In[7]:= get ball.position.y
Out[7]= 2.949999999869974
In[8]:= 5 * 0.7 * 0.7 + 0.5
Out[8]= 2.9499999999999997
In[10]:= get ball.position.y
Out[10]= 1.7004999998062575
In[11]:= 5 * 0.7 * 0.7 * 0.7 * 0.7 + 0.5
Out[11]= 1.7004999999999997
In[13]:= get ball.position.y
Out[13]= 1.0882449997748824
In[14]:= 5 * 0.7^6 + 0.5   (written out longhand in the script)
Out[14]= 1.0882449999999997
```

Three apexes, each matching to about 2×10^{-10} — and each arrived at at the right *moment*, which is the harder half.

Also notice $|dE/E| = 4.636\text{e-}1$. Energy is **not** conserved here, and should not be: $e = 0.7$ means each bounce deliberately throws away 51 % of the kinetic energy. A conservation check is only meaningful when the physics is conservative. This is the first place most people misread the diagnostic.

7.5 Cyclotron motion, and when to reach for BDF

A charge q moving at speed v perpendicular to B travels in a circle of radius $r = m|v|/(|q|B)$ with period $T = 2\pi m/(|q|B)$. The Lorentz force $q\,v \times B$ depends on velocity — which rules out symplectic methods entirely — and is fast compared with anything else in the problem. That is the textbook definition of a *stiff* system, and what `METHOD BDF` is for.

Half a period puts the ion diametrically opposite, so the distance from the origin must be exactly $2r$.

```
In[6]:= get ion.position
Out[6]= [-0.00000000020913167204927863, 1.1111111086955652, 0]
In[7]:= norm(ion.position)
Out[7]= 1.1111111086955652
In[8]:= 2 * (2 * 5 / (3 * 6))
Out[8]= 1.1111111111111112
In[10]:= get ion.position          (after the second half period)
Out[10]= [0.000000000408039691444928, 0.00000000538188465850958 3, 0]
In[11]:= get ion.velocity
Out[11]= [4.999999951563042, 0.000000003672359576156747 4, 0]
```

Right to 8 significant figures; after a full period the ion is back at the origin to 5 parts in a billion with its original velocity restored. The magnetic force does no work, and the speed confirms it.

7.6 Symplectic versus adaptive, over a long run

CVODE Adams controls the *local* error at every step. A **symplectic** method controls something else entirely: it exactly preserves the geometric structure of Hamiltonian mechanics, so the energy error does not accumulate — it oscillates around a constant.

```
In[5]:= run 60 steps 60                          (METHOD ADAMS)
Out[5]= t = 60 (507075 solver steps, 60 snapshots, |dE/E| = 1.459e-8)
In[6]:= energy
Out[6]= -0.20000000291885495

In[12]:= run 60 steps 60      (METHOD SPRK MCLACHLAN_4_4 0.001)
Out[12]= t = 60 (60000 solver steps, 60 snapshots, |dE/E| = 8.957e-11)
In[13]:= energy
Out[13]= -0.20000000001790574
```

The symplectic method took **60,000** steps to the adaptive method's **507,075**, and its energy error is **163 times smaller**. Eight times less work, two orders of magnitude better answer.

The catch, and it is a real one. SPRK needs a *separable* Hamiltonian. A magnetic field breaks that, and so does rotation. The program does not let you get this wrong quietly:

```
Err: SPRK method requires a separable Hamiltonian: magnetic field B must
     be zero (the Lorentz force q v x B is velocity-dependent);
     use METHOD ADAMS or BDF
```

The error names the feature that blocks it. That refusal is why the previous example uses BDF.

7.7 The tennis-racket theorem, measured

A rigid body has three principal axes with three different moments of inertia. Spin it about the largest or the smallest and the motion is stable. Spin it about the **middle** one and it is not: the body flips end over end, over and over. Cosmonaut Vladimir Dzhanibekov noticed this on Salyut 7 in 1985 with a wing nut.

The flip is a genuine instability, so it is exquisitely sensitive to integration error — and yet the angular momentum must be *exactly* conserved throughout, because nothing is applying a torque.

Those two demands pull in opposite directions.

```
In[2]:= angmom
Out[2]= [0.02, 3, 0.02]
In[3]:= energy
Out[3]= 5.148625491836798

racket.orientation.x, sampled every 0.5 from t = 0.5 to t = 4.0:
   0.013359726984815766
   0.009214316848229325
  -0.06928171314906612
  -0.32837233114496406
  -0.4991829732950659
   0.1093367245470612
   0.8230403847701171
   0.9661018228588313

In[20]:= angmom
Out[20]= [0.02, 3, 0.02]
In[21]:= energy
Out[21]= 5.148625488700016
```

The x component of the orientation quaternion — near enough, "how far over has it rolled" — sweeps from 0.013 through negative territory to 0.966. That is the flip. And through all of it, `ANGMOM` reads $[0.02, 3, 0.02]$ at the end exactly as it did at the start: not to eight decimals, the identical printed value.

You can watch this one: `videos/tumbling_racket.html`.

7.8 When energy *should* grow

A magnetised body in a field feels a torque $\tau = (RMR^T)B$. The field is an outside agent: it does work, so kinetic energy grows and angular momentum about the origin is *not* conserved. Half of understanding a conservation diagnostic is knowing when it should fail.

```
In[3]:= new cuboid as rotor { mass = 1, half_extents = [1, 0.4, 0.4],
          magnetic_moment_tensor = [[0, 0, 0.5], [0, 0, 0], [-0.5, 0, 0]] }
In[4]:= energy
Out[4]= 0
In[5]:= angmom
Out[5]= [0, 0, 0]
In[6]:= run 3 steps 30
Out[6]= t = 3 (550 solver steps, 30 snapshots, |dE/E| = 9.928e-1)
In[7]:= energy
Out[7]= 0.9928369509454675
In[8]:= angmom
Out[8]= [0.46022300703213415, 0, 0]
```

Starts at rest, ends spinning. The initial torque is $M \cdot B = (0.5 \cdot 2, 0, 0) = (1, 0, 0)$, and the angular momentum duly grows along x.

The trap. With `[[0, 0.5, 0], [-0.5, 0, 0], [0, 0, 0]]` — a perfectly reasonable-looking

antisymmetric tensor — the *third column* is zero, $M \cdot B$ is the zero vector, and absolutely nothing happens. No error, no warning, just a body that sits there. Multiply the tensor by your field by hand before you blame the solver.

7.9 Newton's cradle

Five identical spheres, four of them touching. Roll one into the line and *only the far one* leaves, at exactly the incoming speed. Momentum alone does not force this — two balls leaving at half speed would conserve momentum fine. It is momentum **and** energy together that pick out the answer.

```
In[8]:= energy
Out[8]= 0.5
In[9]:= momentum
Out[9]= [1, 0, 0]
In[10]:= run 6 steps 600
Out[10]= t = 6 (30 solver steps, 600 snapshots, |dE/E| = 0.000e0,
         4 collision(s))
In[11..14]:= get b0..b3 .velocity
Out[11..14]= [0, 0, 0]   (all four)
In[15]:= get b4.velocity
Out[15]= [1, 0, 0]
In[16]:= energy
Out[16]= 0.5
In[17]:= momentum
Out[17]= [1, 0, 0]
```

Four impulses, and not one bit lost.

7.10 The bullet that does not tunnel

A 4 mm bullet at 100 m/s meets a 5 mm plate. Between output frames the bullet travels a metre — two hundred plate thicknesses. This is precisely the case that defeats frame-sampled collision detection.

```
In[5]:= run 0.1 steps 10
Out[5]= t = 0.1 (10003 solver steps, 10 snapshots, |dE/E| = 0.000e0,
        1 collision(s))
In[6]:= get bullet.position.y
Out[6]= 8.009000000001455
In[7]:= get bullet.velocity.y
Out[7]= 100
In[9]:= contacts
Out[9]= contact0: obj0 <-> obj1 at t = 0.01995500000000067
  point  = [0, 0.0025, 0]
  normal = [-0, 1, -0]  (from obj0 toward obj1)
  approach speed = 100, impulse = 2
```

The bullet had 1.9955 m of gap to close at 100 m/s, so the exact answer is 0.019955. Look also at the step count: **10,003 solver steps** for ten output points. That is the anti-tunnelling cap at work — the solver was told it may not take a step large enough to skip the plate, and it obeyed.

7.11 Lagrange's equilateral three-body solution

Three equal masses at the corners of an equilateral triangle, each moving at the right speed, rotate rigidly forever — one of only a handful of exactly solvable configurations of the three-body problem, found by Lagrange in 1772. The same geometry is why Jupiter has Trojan asteroids. With $G = m = 1$ and side a, $\omega = \sqrt{3Gm/a^3}$.

There are three independent things to check: each body comes home, the centre of mass never moves, and the triangle stays equilateral.

```
In[6]:= energy
Out[6]= -1.4999999999984994
In[7]:= run 3.6275987284684357 steps 400
Out[7]= t = 3.6275987284684357 (1289 solver steps, 400 snapshots,
        |dE/E| = 4.689e-10)
In[8]:=  get a.position
Out[8]=  [0.5773502692574497, 0, -0.000000002562782285431969 3]
In[9]:=  get b.position
Out[9]=  [-0.2886751324104037, 0, 0.5000000013400956]
In[10]:= get c.position
Out[10]= [-0.28867513684704826, 0, -0.49999999877731977]
In[11]:= com
Out[11]= [-7.586524001605236e-16, 0, -2.146431180941969e-15]
In[12]:= norm(a.position - b.position)
Out[12]= 1.0000000001184224
```

After a full revolution all three are back within about 3×10^{-9}. The centre of mass has moved by 2×10^{-15} — five million times less, which is what you would expect, since it is protected by momentum conservation rather than by accuracy. And the side length is still 1 to 1 part in 10^{10}.

Note also `norm(a.position - b.position)`: you can do vector algebra on live simulator fields directly in the language.

7.12 A user-defined constructor, and an inertia tensor

A `DUMBBELL` is two solid spheres joined by a rod, treated as **one rigid body**. Its inertia about the centre of mass is the exact composite: $\frac{2}{5}mr^2$ for each sphere, plus mz^2 by the parallel-axis theorem, plus the rod's own terms. The inertia tensor is where compound-body code usually goes wrong, and the answer is a hand calculation.

```
In[1]:= def barbell(m1 = 1, m2 = 1, m_rod = 0.5, r1 = 0.3, r2 = 0.3,
                    rod_r = 0.05, len = 2) {
  new dumbbell as bar { m1 = m1, m2 = m2, m_rod = m_rod, r1 = r1,
                        r2 = r2, rod_radius = rod_r, length = len }
}
Out[1]= function barbell(7 parameter(s)) defined - 1 body line(s)
In[3]:= barbell(2, 2, 0, 0.3, 0.3, 0.05, 2)
Out[3]= obj0 as bar
In[4]:= list
Out[4]= obj0: dumbbell r1=0.3 r2=0.3 rod_r=0.05 len=2, mass=4,
        charge=0, pos=[0, 0, 0]
In[9]:= get bar.inertia_tensor
Out[9]= [[4.144, 0, 0], [0, 4.144, 0], [0, 0, 0.144]]
```

```
In[10]:= 2 * (0.4 * 2 * 0.3 * 0.3 + 2 * 1 * 1)
Out[10]= 4.144
In[11]:= 2 * (0.4 * 2 * 0.3 * 0.3)
Out[11]= 0.144
```

4.144 and 0.144, exactly. The off-diagonal entries are exactly zero, as they must be for a body symmetric about its z axis. Every round shape in this program — torus, disk, cylinder, dumbbell — is symmetric about its **local z axis**; that is the convention.

7.13 Geometry a bounding sphere gets wrong

An axis-aligned torus with outer radius 2 exactly inscribes a cube of inner side 4. Tilt that same torus so its axis points along $(1,1,1)/\sqrt{3}$ and its extent along each coordinate axis *drops* to $1.5\sqrt{2/3} + 0.5 \approx 1.7247$, well inside the walls.

A bounding sphere would say the opposite: the torus's bounding sphere has radius 2 no matter how you turn it. Getting this right requires the actual **support function** of the shape — the farthest point of the body in a given direction — evaluated in the rotated frame.

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 - six static walls obj0..obj5 with
        inverse_mass = 0
In[7]:= new torus as tilt { mass = 1, ring_radius = 1.5,
          tube_radius = 0.5, position = [0, 0, 0],
          orientation = [0.8880738339771154, -0.32505758367186816,
                         0.32505758367186816, 0] }
In[8]:= run 1 steps 10
Out[8]= t = 1 (4 solver steps, 10 snapshots, |dE/E| = 0.000e0)
In[10]:= get system.collisions
Out[10]= 0
In[11]:= 1.5 * sqrt(2 / 3) + 0.5
Out[11]= 1.724744871391589
```

Zero collisions. The clearance ($2 - 1.7247 = 0.2753$ per side) is exactly what the geometry predicts. Note also the `BOX` machinery: six wall slabs, created automatically, with `inverse_mass = 0` — *infinite* mass, so they absorb any impulse without ever moving.

7.14 The infinite square well, solved in the language

A particle confined to a box of width L has energies $E_n = n^2\pi^2\hbar^2/(2mL^2)$. `QM GRID` pins the wavefunction to zero just outside the domain, which *is* an infinite square well — the program is solving the real problem, not a special case of it.

```
In[1]:= qm grid 0 1 800
In[2]:= qm potential zero
In[5]:= qm states 4
Out[5]= 4 lowest bound state(s):
  E[0] =  4.934795874467
  E[1] = 19.739107587630
  E[2] = 44.412707408131
  E[3] = 78.955215788088
In[6..9]:= n^2 * pi * pi / 2  for n = 1..4
```

```
Out[6]=  4.934802200544679
Out[7]= 19.739208802178716
Out[8]= 44.41321980490211
Out[9]= 78.95683520871486
```

The agreement is to about 1.3 parts in a million, and it gets *worse* for higher levels. That is not sloppiness: it is the finite-difference approximation to the second derivative, whose error grows with the curvature of the state, and which shrinks as h^2 when you refine the grid.

A physical cross-check on the loaded state:

```
In[11]:= qm norm
Out[11]= 1.000000000000345
In[12]:= qm position
Out[12]= 0.500000000001550
In[14]:= qm prob 0 0.5
Out[14]= 0.499999999998032
```

Total probability 1, mean position dead centre, exactly half the probability in each half of the box — all to 12 digits. The *eigenvalue* carries discretisation error; the *state* is properly normalised and properly symmetric.

7.15 Tunnelling, and watching a grid converge

A particle with energy E below a barrier of height V_0 and width a still gets through, with

$$T = \frac{1}{1 + \dfrac{V_0^2 \sinh^2(\kappa a)}{4E(V_0 - E)}}, \qquad \kappa = \frac{\sqrt{2m(V_0 - E)}}{\hbar}.$$

With $V_0 = 4$, $a = 1$, $E = 2$, $\hbar = m = 1$ that is $0.07065082485\ldots$

Transmission depends on the barrier width *exponentially*, so a grid that gets the width slightly wrong gets T badly wrong. That makes it an unusually sharp probe of the discretisation.

```
In[7]:= 1 / (1 + v0 * v0 * s * s / (4 * e * (v0 - e)))
Out[7]= 0.07065082485316446

qm grid -30 30  4000 -> T = 0.069368404960, T + R = 1.000000000000
qm grid -30 30 16000 -> T = 0.070328037206, T + R = 1.000000000000
qm grid -30 30 64000 -> T = 0.070569990790, T + R = 1.000000000000
```

points	T	**error**
4,000	0.069368	1.3×10^{-3}
16,000	0.070328	3.2×10^{-4}
64,000	0.070570	8.1×10^{-5}

The error falls by a factor of ~ 4 each time the grid is refined by a factor of 4 — first-order convergence, exactly what a staircase-sampled barrier gives you. **This is what "discretisation error" looks like from the outside**, and it is how you tell it from a bug: a bug does not converge.

Notice too that $T + R = 1.000000000000$ on every grid. Probability is conserved exactly regardless of how coarse the grid is, because that is a structural property of the transfer matrix, not an accuracy property. The two kinds of correctness are independent.

Above the barrier there are resonant energies where it becomes perfectly transparent:

```
In[19]:= qm scan 4.1 20 200
Out[19]= 200 energies in [4.1, 20] (0 refused), T from 3.535e-1 to 1.000e0
  1 resonance(s), strongest first:
    E = 8.893969849   T = 0.999992036
```

7.16 The special-function library, checked by identities

The way to test such a library is not to compare against a table (tables have typos) but against **identities** — relations a wrong implementation cannot satisfy by accident.

```
In[1]:= bessel_j(0, 2.404825557695773)
Out[1]= 0
In[2]:= bessel_j_nu(1, 3) * bessel_y_nu(0, 3)
        - bessel_j_nu(0, 3) * bessel_y_nu(1, 3)
Out[2]= 0.21220659078919374 + 0i
In[3]:= 2 / (pi * 3)
Out[3]= 0.2122065907891938
In[4]:= legendre_p(5, 1)
Out[4]= 1
In[5]:= legendre_p(5, 0.906179845938664)
Out[5]= -1.7763568394002506e-16
In[6]:= gamma_z(0.5) * gamma_z(0.5)
Out[6]= 3.1415926535898397 + 0i
In[7]:= pi
Out[7]= 3.141592653589793
In[8]:= sph_harm_real(1, 0, 0, 0)
Out[8]= 0.48860251190291987
In[9]:= sqrt(3 / (4 * pi))
Out[9]= 0.4886025119029199
In[10]:= clebsch_gordan(0.5, 0.5, 0.5, -0.5, 1, 0)
Out[10]= 0.7071067811865475
In[11]:= sqrt(0.5)
Out[11]= 0.7071067811865476
In[12]:= gauss_legendre(4)
Out[12]= [[-0.8611363115940526, -0.3399810435848563,
            0.3399810435848563,  0.8611363115940526],
          [ 0.34785484513745374, 0.6521451548625461,
            0.6521451548625461,  0.34785484513745374]]
```

Line by line: J_0 at its first zero is a clean 0. The Wronskian identity $J_{\nu+1}Y_\nu - J_\nu Y_{\nu+1} = 2/(\pi x)$ matches to 16 digits — a genuinely strong test, since it couples four different function evaluations. $P_5(1) = 1$ exactly, and 0.906179845938664 really is one of its roots (residual one bit). $\Gamma(\frac{1}{2})^2 = \pi$ to 14 digits, via the *complex* gamma evaluated on the real axis. The spherical harmonic Y_1^0 at $\theta = 0$ is $\sqrt{3/4\pi}$ to 15 digits. Two spin-$\frac{1}{2}$ particles coupling to the $m = 0$ triplet have Clebsch–Gordan coefficient $1/\sqrt{2}$ to the last bit. And the 4-point Gauss–Legendre rule reproduces the classical nodes and weights, whose sum is 2 — the length of $[-1, 1]$.

One design decision worth knowing. Where an argument is an integer order, a fractional value is refused rather than truncated:

```
In[1]:= hermite_h(2.5, 1)
Err[1]: hermite_h(): argument 1 must be a whole number
        (an integer order), got 2.5
```

Truncating would have returned a confident, wrong number with no way for you to notice. Angular momenta, on the other hand, *may* be half-integers, so the Wigner and Clebsch–Gordan routines take plain numbers.

7.17 A pendulum as a constraint, not a force

The small-amplitude period is $T = 2\pi\sqrt{L/g}$; released 0.02 rad off vertical with $L = 1$, $g = 9.81$ that is 2.0060666807106475 s, and after exactly one period the bob must be back where it started. The rod is not a force but a geometric condition, and the only way to hold it exactly is to solve the DAE.

```
In[4]:= new sphere as pivot { mass = 1, radius = 0.02,
                              position = [0, 0, 0], inverse_mass = 0 }
In[5]:= new sphere as bob   { mass = 1, radius = 0.05,
          position = [0.019998666693333084, -0.9998000066665778, 0] }
In[6]:= constrain pivot bob
Out[6]= constraint0: obj0 <-> obj1 held at 1
        (METHOD IDA is required to integrate it)
In[8]:= method ida
Out[8]= method = IDA (constrained DAE, GGL index-2)
In[9]:= run 2.0060666807106475 steps 200
Out[9]= t = 2.0060666807106475 (339 solver steps, 200 snapshots,
        |dE/E| = 2.530e-12)
In[10]:= get bob.position
Out[10]= [0.019998666320588818, -0.9998000066740419, 0]
In[11]:= constraints
Out[11]= constraint0: obj0 <-> obj1 length 1
         (currently 1.0000000000000082)
```

The bob came back to within 3.7×10^{-10} after a full period, and the rod reads 1.0000000000000082 — eight parts in 10^{15}, one bit, after 339 solver steps. That is not luck: the formulation carries *both* the rod's length and its rate of change as equations. The cheaper scheme that constrains only the acceleration lets the length drift quadratically, and you find out much later.

The `inverse_mass = 0` on the pivot is what makes it a pendulum rather than two bodies tumbling about their shared centre of mass joined by a stick.

7.18 Where it rests, and what the answer depends on

A pendulum released anywhere comes to rest hanging straight down, one rod-length below the pivot:

```
In[7]:= equilibrium
Out[7]= equilibrium found in 17 Newton iteration(s), 67 residual
        evaluation(s); largest net force on any free body =
```

```
        7.459152323898993e-13, worst |g| = 1.9317880628477724e-14
In[8]:= get bob.position
Out[8]= [0.00000000000007352624797250O6, -0.9999999999999807, 0]
```

Released 57° off vertical, the bob lands straight down to 13 digits. Now the derivative, on a free body, where $\partial y/\partial g = T^2/2 = 4.5$ at $T = 3$:

```
In[15]:= sensitivity 3 "gravity.y" "mass 0"
Out[15]= t = 3 (CVODES, 129 solver steps)
d/d(gravity.y):
  obj0 position [0, 4.500000056696235, 0]
d/d(mass 0):
  obj0 position [0, 0, 0]
In[16]:= get stone.position
Out[16]= [3.000000000000001, -44.14500000000045, 0]
```

4.500000056696235 against an analytic 4.5 — 1.3 parts in 10^8, carried alongside a trajectory that itself landed on -44.14500000000045 where $-\frac{1}{2} \cdot 9.81 \cdot 9 = -44.145$.

And the second derivative is **exactly zero**. In uniform gravity every mass accelerates equally, so the trajectory does not depend on the mass at all. A finite-difference estimate would have returned some small number and left you guessing whether it was noise or physics. The sensitivity equations return the zero.

7.19 A door on a hinge

A hinge fixes a point *and* an axis, leaving one freedom. A hinged rigid body is a **compound** pendulum: its small-amplitude period is $T = 2\pi\sqrt{I_{\text{pivot}}/(mgd)}$ with $I_{\text{pivot}} = I_{\text{com}} + md^2$ — the moment of inertia about the pivot, not just the distance to the centre of mass. The point-mass formula is wrong by 15 % for the slab below.

```
In[4]:= new sphere as jamb { mass = 1, radius = 0.02,
                             position = [0, 0, 0], inverse_mass = 0 }
In[5]:= new cuboid as door { mass = 1, half_extents = [0.2, 0.4, 0.2],
          position = [0.0199986666933331, -0.9998000066665778, 0] }
In[6]:= hinge jamb door [0, 0, 1]
In[8]:= method ida
Out[8]= method = IDA (constrained DAE, GGL index-2)
In[9]:= run 1 steps 10
Out[9]= t = 1 (70 solver steps, 10 snapshots, |dE/E| = 1.613e-9)
In[10]:= constraints
Out[10]= constraint0: hinge obj0 <-> obj1, 5 row(s)
worst |g| = 2.73750133672479e-10, worst |g_dot| = 2.3769240437118873e-9
In[11]:= get door.angular_momentum
Out[11]= [0, 0, 0.003742219622967182]
```

The angular momentum is $[0, 0, 0.0037]$ — *exactly* zero on x and y. The two rows a hinge has over a ball joint are the ones that forbid those axes, and they are doing their job to the last bit. The joint is held to 2.7×10^{-10} after 70 solver steps, and energy to 1.6 parts in 10^9; run one compound period and the slab returns to within 3×10^{-8}.

The `inverse_mass = 0` on the jamb is what makes it a door: an anchor, immovable, absorbing whatever the hinge pulls with.

Two limits worth knowing. Orientation joints are integrated *from rest* — a body already turning at t_0 is refused by name — and they carry a tolerance floor of `rtol` $= 10^{-6}$, because the index-2 system has an accuracy ceiling no tolerance can push past. Both are refusals rather than guesses: a run that looks like it worked and is not the mechanism you described is the worst possible outcome.

8 Watching it: the scene window and browser videos

8.1 The live window

Type `SCENE CREATE` and a browser tab opens. That tab *is* the simulator's window: it draws every body on a 3-D canvas, with a toolbar (Start, Pause, Stop, Reverse, Reset, single-step, a *dt* box, zoom, grid/trails/labels toggles) and a status bar showing the playback mode, the time, *dt*, the total energy, the body count, the camera and the frame rate.

Behind it is a web server written on the Rust standard library — including the HTTP parsing, the WebSocket handshake, the SHA-1 and the base64. No dependency, and the page itself is vanilla JavaScript and canvas with no CDN fetches.

Two things about it are worth knowing. **The window evolves its own copy of the system**: typing `STEP` in the notebook does not move the window, and pressing Start in the window does not move the notebook. That isolation is what lets the window run without ever locking the language. And **Reverse is replay, not negative-time integration**: the playback keeps a ring of snapshots and walks it backward, returning to $t = 0$ bit-identically.

8.2 Recorded videos

A live window needs a live program. For something you can send to someone else, record it:

```
cargo build --release -p posim
tools/record_video.py videos/scenes/kepler_ellipse.posim \
     -o videos/kepler_ellipse.html --frames 360 --dt 0.02 \
     --title "Kepler orbit, e = 0.6"
```

The result is a single HTML file: the frames embedded as data plus a canvas player. It fetches nothing, so it works from `file://` on a machine with no network, forever. The recorder drives `posim --machine` and asks it to `STEP` — **every advance is a real SUNDIALS step**. The tool is a camera, not a physics engine.

file	what to watch	measured
`kepler_ellipse.html`	the speed swinging between perihelion and aphelion	$\lvert dE\rvert/E = 9.8 \times 10^{-8}$, $\lvert dL\rvert/\lvert L\rvert = 1.3 \times 10^{-7}$
`tumbling_racket.html`	the tennis-racket flip, seen from outside	$\lvert d\lvert L\rvert\rvert/\lvert L\rvert = 0$ **exactly**; $\lvert dE\rvert/E = 6.4 \times 10^{-9}$
`box_of_shapes.html`	a cylinder, a disk and a cuboid in a rigid `BOX` 4; gold arrows are the analytic contact normals, sized by impulse	36 collisions, $\lvert dE\rvert/E = 3.4 \times 10^{-16}$

The player has Play/Pause (or Space), frame stepping (or ←→), a scrub bar, speed from 0.25× to 4×, drag to orbit, wheel to zoom, and toggles for trails, labels and contact arrows. The corner readout shows the frame number, t, E, $|P|$, $|L|$ and the collision count *for the frame you are on* — so you can stop on the moment something looks wrong and read the conserved quantities off it.

9 How you know the numbers are right

9.1 The solver library against its C original

`sundials_rs/` ships the upstream SUNDIALS example programs, translated. Each one is run and its output **diffed byte-for-byte** against the output of the original C program. The results, the divergences, and the reasons for each divergence are in `sundials_rs/VERIFICATION.md`, `sundials_rs/differences/` and `sundials_rs/c-results/`.

Two consequences of taking this seriously: printed floating-point values go through helpers that reproduce C's `printf` conversions exactly (Rust's own `{:e}` formats exponents differently), and `pow` was made host-independent so that at least that function cannot vary between machines.

9.2 The simulator against closed-form physics

Six example programs, each printing `SUCCESS` or `FAILURE` and exiting nonzero if it fails: `kepler_orbit`, `outer_solar_system`, `tumbling_body`, `charged_in_b_field`, `newtons_cradle`, `bouncing_ball_restitution`.

9.3 The test suite

603 tests: 40 library, 19 collision, 9 conservation, 109 language, 92 quantum, 233 special-function, 11 vendored identities and 55 documentation examples that are compiled and run as written. The collision tests in particular are analytic rather than regression-style: they assert the time of impact against a closed form, not against last week's output.

9.4 The notebooks

`dynamic_notebooks/` holds 59 runnable sessions, including 34 problems from Routh's *Dynamics of a Particle* (1898) and *Dynamics of a System of Rigid Bodies* (1905). Each derives its closed-form answer in its own header and then checks the integrator against it — so the numbers in the catalogue are measured, not quoted.

9.5 And for the 7.8.0 upgrade specifically

Everything above was run against **both** the old 7.7.0 engine and the new 7.8.0 one, and the outputs diffed. They are identical byte for byte. See `PORT_7.8.0_PROVENANCE.md` and `evidence/port-7.8.0/`.

10 When something goes wrong

Errors name the column, the field, or the feature at fault, and never abort your session.

symptom	what is actually happening
$\lvert dE/E \rvert$ is huge and you expected conservation	Check `system.g_constant` (§3.2), the restitution of your bodies, and whether a field is doing work.
`RUN` overshoots where you meant to stop	`RUN` takes a **duration**.
bodies pass through each other	You did not `COLLIDE ON`.
`SPRK method requires a separable Hamiltonian`	A magnetic field, a magnetic moment, an external torque or a spinning body is present. The message names which. Use `METHOD ADAMS` or `BDF`.
`no object obj7` after a `DEL`	Deleting renumbers. Use `NEW ... AS` *name*.
`unknown name 'x'`	A bare identifier resolves at execution time. Bind it with `LET`, pass it as a function parameter, or use *name*`.`*field*.
a magnetised body will not spin	Multiply your moment tensor by your field by hand — you probably have a zero column.
a quantum result is off by a fraction of a percent	Refine the grid and see whether it converges. Discretisation error converges; a bug does not.
`probability accumulating near the wall`	Your wavepacket has reached the domain edge and is reflecting. Widen the grid or add `QM ABSORB`.
the scene window does not open	Set `POSIM_NO_BROWSER=1` if you are headless, or open the printed `http://127.0.0.1:<port>/` yourself.

Anything the language accepts is documented in `grammar.md` — every command, every field, every builtin, with the complete EBNF grammar and a further eighteen worked examples.

11 Where everything lives

path	what
`posim/`	the command language, the graphics window, the machine protocol
`physical_object/`	the physics: the union type, collisions, all time integration
`quantum/`	1-D, 2-D and 3-D quantum mechanics
`special_functions/`	Bessel, Legendre, Hankel, Airy, gamma, Wigner, quadrature, eigenproblems
`sundials_rs/`	the pure-Rust SUNDIALS 7.8.0 engine (vendored, read-only)
`jupyter/`	the JupyterLab kernel
`dynamic_notebooks/`	59 runnable sessions, incl. 34 Routh problems
`scripts/solveit/`	the eighteen examples in §7
`scripts/collisions/`	twelve documented collision scripts
`videos/`	recorded browser videos; `videos/scenes/` the scripts behind them
`tools/`	the index builder, the verifiers, `record_video.py`
`evidence/port-7.8.0/`	the logs behind every claim in §9.5

document	for
`SolveIt.md` / `.pdf`	this one: the complete solution guide
`grammar.md` / `.pdf`	every command, the full EBNF, 18 more worked examples
`ARCHITECTURE.md`	the pinned cross-module contracts
`PORT_7.8.0_PROVENANCE.md`	what the 7.8.0 upgrade changed, and the evidence it changed nothing else
`collision_detection.md` / `.pdf`	the collision science, with 12 example scripts
`scene_info.md` / `.pdf`	the graphics window, and a survey of seven other simulators
`physical_object_simulator.md` / `.pdf`	the predecessor user guide, with 14 further examples
`special_functions.md`	the special-function library in detail
`CLAUDE.md`	the working rules for anyone modifying this repository
`index_of_entities.html`	a browsable catalogue of every named entity in the repository

Everything above was produced by the program in this repository. Every command shown can be typed. Every number shown can be reproduced.